

ITERATION PREPARATION

ITERATION 2 PREPARATION

Ready to Begin Cycle 2

PREP-20260114_204608

ITERATION:	Iteration 2
DATE:	2026-01-14 20:46:08
STATUS:	Ready to Begin

SCIENTIFIC METHOD - PREPARE → EXECUTE → OBSERVE → ITERATE

PDF Generation Improvements - Iteration 2 Preparation

Prepared: 2026-01-14 20:46:08

Cycle: 2

Status: Ready to Begin

STARTING CONDITIONS (SAME AS CYCLE 1)

System State

- **PDF Generation:** Using BriefDocument with professional cover pages
- **Code Examples:** Automatic extraction and insertion
- **Markdown Conversion:** Enhanced with syntax highlighting support
- **Title Handling:** Proper generation and escaping
- **Verdict Classification:** Dynamic based on proof results

Test Cases (Same as Cycle 1)

1. Generate PDF from case file with multiple claims
2. Verify code examples are added automatically
3. Check markdown conversion with code blocks
4. Verify title generation and escaping
5. Test verdict-based cover page classification

TARGET IMPROVEMENTS (FROM CYCLE 1 OBSERVATIONS)

High Priority

1. **Improve Fallback Markdown Conversion**
2. Better regex patterns for code blocks
3. Improved table conversion
4. Better header handling
5. Preserve formatting when markdown library unavailable

Medium Priority

1. **Enhance Code Example Extraction**

2. Scan entire document for code references
3. Extract from multiple sections (not just evidence)
4. Include file paths and line numbers
5. Better code block detection

Low Priority

1. **Better Filename Generation**
2. Smarter truncation (preserve important words)
3. Use hash for very long titles
4. Better handling of special characters in filenames
5. **Error Handling**
6. Better error messages if PDF generation fails
7. Handle PDF opening failures gracefully
8. Clear error reporting

IMPLEMENTATION PLAN

Step 1: Enhance Fallback Markdown

- Improve regex patterns for code blocks
- Add table conversion regex
- Better header detection
- Preserve inline formatting

Step 2: Improve Code Example Extraction

- Scan full document content
- Extract from all sections
- Include context (file paths, line numbers)
- Better code block formatting

Step 3: Smarter Filename Generation

- Implement word-preserving truncation
- Add hash fallback for very long titles
- Test with various special characters

Step 4: Error Handling

- Add try/except blocks
- Clear error messages
- Graceful degradation

SUCCESS CRITERIA

Must Have

- ☒ All current functionality works
- ☒ Fallback markdown handles common features
- ☒ Code examples extracted from all sections

Should Have

- ☒ Better error messages
- ☒ Smarter filename generation
- ☒ Enhanced code example context

Nice to Have

- ☒ Hash-based filenames for very long titles
- ☒ Advanced markdown features in fallback
- ☒ Code example line number extraction

NOTES

- Continue using BriefDocument for professional output
 - Maintain backward compatibility
 - Test with various case file formats
 - Document any new utility functions created
-

Ready to begin Iteration 2

Prepared: 2026-01-14 20:46:08